

MySQL Stored Procedures — Scenario-Based Super Guide

1. What is a Procedure?

A **procedure** in MySQL is:

- A **saved SQL program** inside the database.
- You can **run it anytime** without retyping all the code.
- It can **take inputs** (like giving instructions) and/or **return outputs** (like giving results back).

💡 **Analogy:**

It's like telling your pizza shop's oven,

"Cook me a Pepperoni with extra cheese" —

You give the oven instructions (**IN**) and later get the finished pizza (**OUT**).

2. Types of Parameters

Type	Goes In?	Comes Out?	Example Use
IN	☒ Yes	☒ No	Pass customer ID to find their orders
OUT	☒ No	☒ Yes	Get total number of sales
INOUT	☒ Yes	☒ Yes	Give points value, update it, get new value

3. Real-World Story

Imagine we own “**Pizza Palace**”.

We have:

- **Customers** table
- **Orders** table
- We use procedures to help staff and managers quickly get answers.

4. Basic → Advanced Scenarios

Scenario 1 — Basic (IN parameter)

Question:

A cashier wants to see all orders for a specific customer.

Create a procedure that takes **customer ID** as input and shows their orders.

Solution:

```
DELIMITER $$

CREATE PROCEDURE ShowOrdersForCustomer(IN custId INT)
BEGIN
    SELECT * FROM Orders WHERE customer_id = custId;
END$$

DELIMITER ;

-- Run it:
CALL ShowOrdersForCustomer(3);
```

What happens:

- 3 (customer ID) goes **into** the procedure.
- Procedure runs the query.
- Result: all orders for customer #3 are shown on screen.

Scenario 2 — Medium (OUT parameter)

Question:

The manager wants to know **how many orders were placed today**.

Make a procedure that calculates the total and sends it back.

Solution:

```
DELIMITER $$

CREATE PROCEDURE CountTodayOrders(OUT total INT)
BEGIN
    SELECT COUNT(*) INTO total
    FROM Orders
    WHERE DATE(order_date) = CURDATE();
END$$
```

```

END$$

DELIMITER ;

-- Run it:
CALL CountTodayOrders(@orders_today);
SELECT @orders_today;

```

What happens:

- No input is given.
- Procedure counts today's orders.
- The number is stored in @orders_today (an **OUT** variable).

Scenario 3 — Advanced (INOUT parameter)

Question:

Pizza Palace has a loyalty program.
 When a customer orders, they get **20 bonus points**.
 Write a procedure that:

1. Takes their **current points** as input.
2. Adds 20.
3. Updates the database.
4. Returns the **new total points**.

Solution:

```

DELIMITER $$

CREATE PROCEDURE AddLoyaltyPoints(IN custId INT, INOUT points INT)
BEGIN
    -- Add bonus
    SET points = points + 20;

    -- Update table
    UPDATE Customers
    SET loyalty_points = points
    WHERE customer_id = custId;
END$$

DELIMITER ;

-- Run it:
SET @pts = 50; -- current points
CALL AddLoyaltyPoints(3, @pts);
SELECT @pts; -- Now shows 70

```

What happens:

- `custId = 3` (IN) and `points = 50` (INOUT) are sent in.
 - Procedure adds 20 points → 70.
 - Updates the DB.
 - Returns the new total in the same `@pts` variable.
-

Scenario 4 — Pro Level (Mix of IN + OUT)

Question:

The manager wants **total sales amount for a specific date**.

Write a procedure that:

1. Takes a **date** as input (IN).
2. Returns the total sales as output (OUT).

Solution:

```
DELIMITER $$

CREATE PROCEDURE GetSalesByDate(IN salesDate DATE, OUT totalSales
DECIMAL(10,2))
BEGIN
    SELECT SUM(amount) INTO totalSales
    FROM Orders
    WHERE DATE(order_date) = salesDate;
END$$

DELIMITER ;

-- Run it:
CALL GetSalesByDate('2025-08-12', @sales);
SELECT @sales;
```

What happens:

- You give it a **date** (IN).
 - Procedure calculates the sum of sales for that date.
 - Result is stored in `@sales` (OUT).
-

5. How to Remember IN vs OUT vs INOUT

💡 Simple Traffic Rule:

- **IN** → Data goes **into** the procedure.
- **OUT** → Data comes **out** from the procedure.

- **INOUT** → Data goes in, **changes**, then comes out.
-

6. Best Practices

- Use **clear parameter names** (`custId`, `salesDate`) for readability.
 - Keep one main purpose per procedure.
 - Always test with **small data** first.
 - Use `DELIMITER` to avoid syntax errors.
-